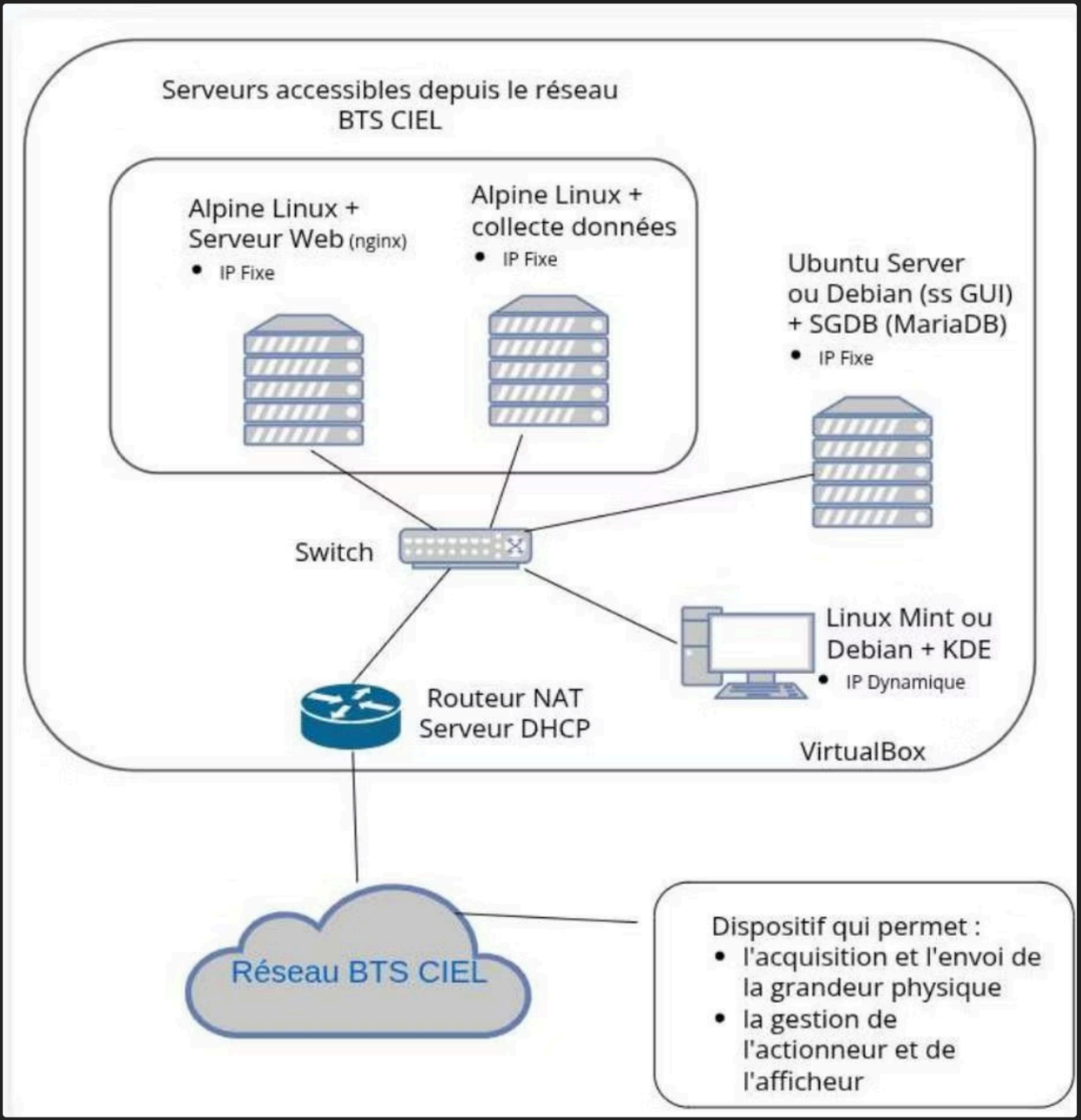
Système de Télémetrie MQTT & MariaDB

Projet 02 : Système de Gestion de l'Éclairage (ECLAIRAGE)

Étudiant : Sita Alexandre | **Date :** 07 Avril 2026 | **Statut :** En cours d'intégration finale

Made with GAMMA

Synoptique Réseau



Architecture centrée sur un **Switch** interconnectant :

- **Alpine Linux** + nginx (IP Fixe)
- **Alpine Linux** + collecte données (IP Fixe)
- **Ubuntu/Debian** + MariaDB (IP Fixe)
- **Linux Mint/Debian** + VirtualBox (IP Dynamique)
- **Routeur NAT** + Serveur DHCP
- **Réseau BTS CIEL** (cloud)

Le dispositif permet l'acquisition de la grandeur physique, la gestion de l'actionneur et de l'afficheur.

Made with GAMMA

PHASE 1

Configuration du Broker

L'unité **Alpine Linux** est configurée comme point central (Broker) pour la réception des messages du Micro:bit ou de la Raspberry Pi.

Installation

`apk update && apk add mosquitto mosquitto-clients`

Configuration

Édition de `/etc/mosquitto/mosquitto.conf` : `listener 1883` et `allow_anonymous true`

Démarrage

`rc-service mosquitto start` et `rc-update add mosquitto`

Made with GAMMA

PHASE 2

Connectivité Réseau

Résolution du problème **Network is unreachable** qui bloquait l'installation des paquets sur Ubuntu Server.

1 Diagnostic

Conflit identifié avec le "Mobile Hotspot" de l'hôte Windows

2 Résolution

Passage de la carte réseau en mode **NAT** sur VirtualBox

3 Validation

`sudo dhclient -v enp0s3` — ping réussi vers `8.8.8.8`

Made with GAMMA

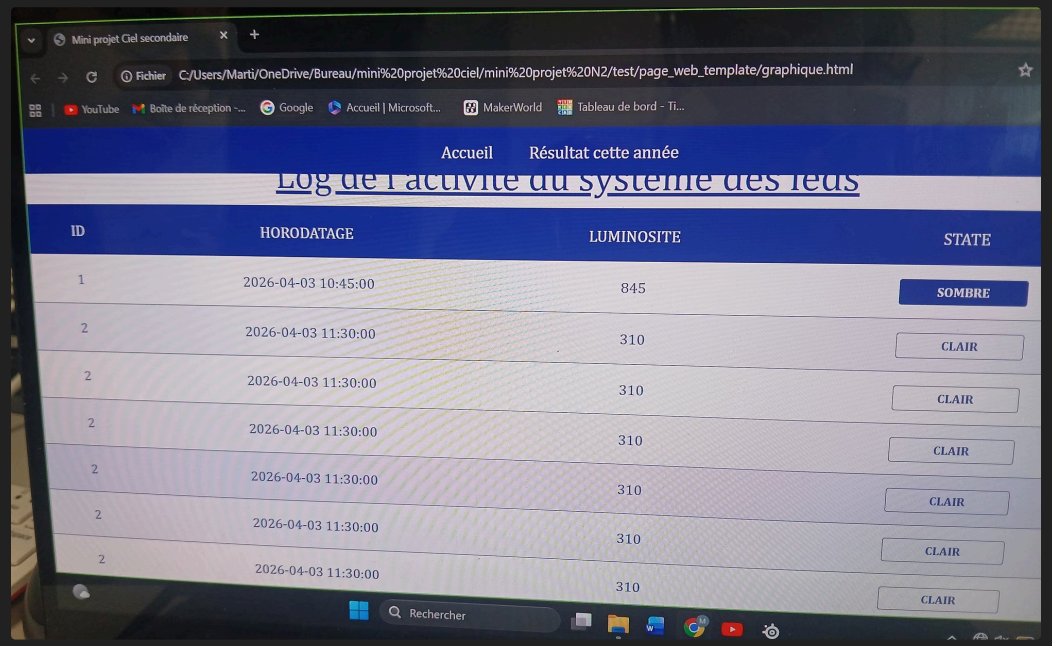
PHASE 3

Base de Données MariaDB

Structure SQL pour l'archivage des données de luminosité. **Erreur 1064** corrigée en séparant les instructions de gestion de base de données de la création de table.

```sql
CREATE DATABASE ECLAIRAGE;
USE ECLAIRAGE;

CREATE TABLE log_sistema (
  id INT AUTO_INCREMENT PRIMARY KEY,
  horodatage DATETIME DEFAULT CURRENT_TIMESTAMP,
  luminosite INT NOT NULL,
  state VARCHAR(10) NOT NULL
);
```



Made with GAMMA

PHASE 4

Script Passerelle Python

Liaison entre le Broker MQTT et la base MariaDB. Corrections majeures appliquées.

API Callback

Mise à jour vers **v2.1.0** pour éviter les avertissements de dépréciation

Syntaxe

Correction des erreurs d'instanciation (`mqtt.Client()`) et de nommage de fonctions

Logique métier

État **"SOMBRE"** si valeur > 500, sinon **"CLAIR"**

Made with GAMMA

Code Source – Gateway Python

```python
import paho.mqtt.client as mqtt
import mysql.connector

MQTT_BROKER = "192.168.x.x"
MQTT_TOPIC = "votre/topic/luminosite"

def on_message(client, userdata, msg):
 payload = msg.payload.decode('utf-8')
 try:
 db = mysql.connector.connect(
 host="localhost", user="root",
 database="ECLAIRAGE")
 cursor = db.cursor()
 valeur = int(payload)
 etat = "SOMBRE" if valeur > 500 else "CLAIR"
 cursor.execute(
 "INSERT INTO log_sistema (luminosite, state) VALUES (%s, %s)",
 (valeur, etat))
 db.commit()
 db.close()
 except Exception as e:
 print(f"Erreur : {e}")

client = mqtt.Client(
 protocol=mqtt.MQTTv311,
 callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_message = on_message
client.connect(MQTT_BROKER, 1883)
client.subscribe(MQTT_TOPIC)
client.loop_forever()
```

Made with GAMMA

PHASE 5

Installation des Dépendances

01

Python 3 & pip

```
sudo apt install python3 python3-pip -y
```

02

Client MQTT

```
pip install --upgrade paho-mqtt
```

03

Connecteur MySQL

```
pip install mysql-connector-python
```

Made with GAMMA

État d'Avancement

5

Phases réalisées

Broker, réseau, BDD, script et dépendances

3

Composants prêts

Broker accessible, BDD structurée, script débogué

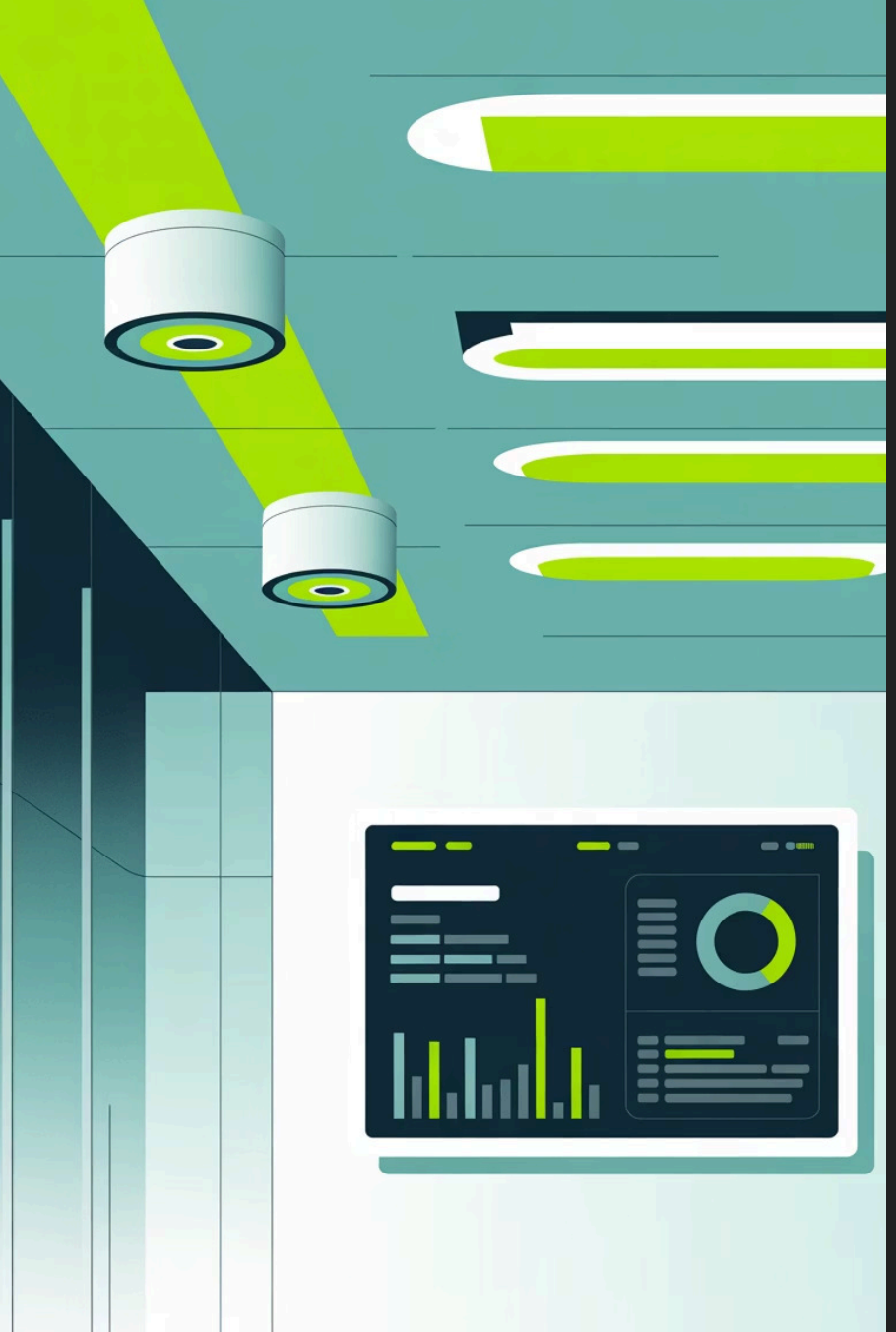
1

Étape restante

Validation end-to-end avec capteur réel

Prochaine étape : Validation finale de la communication de bout en bout pour assurer que les données réelles du capteur sont correctement enregistrées dans la table `log_sistema`.

Made with GAMMA



Conclusion

Le projet a atteint une **maturité technique stable**. L'infrastructure de télémétrie MQTT vers MariaDB est opérationnelle et prête pour le déploiement final.

Broker

Alpine Linux + Mosquitto configuré et accessible

Base de données

MariaDB avec table `log_sistema` structurée

Script Python

Gateway débogué, optimisé et prêt pour la production

Made with GAMMA